

ECE 379K Lab 1 Report

Syed Rayaan Hasan (srh3787), Tu Duong (qd867)

Note: We used Lonestar6 for testing and benchmarking instead of Frontera due to Lonestar6 compiling and running TSan (see lscpu details and TSan output at the bottom of the page)

Part 1:

Why does `size()` need a lock?

Although `size()` only reads the map, concurrent inserts and erases modify the state it reads. Without synchronization, those accesses create a data race and undefined behavior. Taking the same mutex as the other operations prevents concurrent modifications and returns an exact size at a point during the call.

Why does the interface return copies instead of references or iterators?

A reference or iterator would let the caller access internal map data after the method releases its lock. Another thread could then update or erase that entry, causing a data race or invalidating the reference or iterator. Copying the value while holding the lock gives the caller an independent result that remains valid after unlocking.

Validation

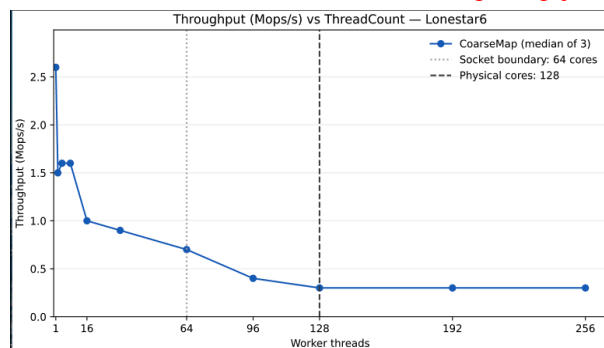
Both test programs passed the correctness tests, and TSan ran on a Lonestar6 compute node, with no compiler warnings or reported data races.

Part 2:

The Throughput vs. ThreadCount curve was measured on Lonestar6, while the counter table was measured on Frontera.

However, their CPU similarity allowed counters to support the contention explanation.

Prediction: Throughput would peak at one or two threads, then generally decline as contention increased. I'm expecting a possible drop when the second socket entered and low or declining throughput under oversubscription.



Observation

Measurements were done on a Lonestar6 compute node with two AMD EPYC 7763 sockets, 64 physical cores per socket, and SMT disabled. The provided sweep restricted execution to CPUs in CPU-number order, filling the first socket before using the second. Each point reports the median of 3 runs.

Throughput declines from **0.9Mops/s at 32 threads** to **0.7Mops/s at 64 threads**, where the first socket is full. At **96 threads** (the first sampled point using both sockets), it falls to **0.4Mops/s**, a >40% reduction from 64-thread throughput. The three-run ranges at 64 threads (0.7-0.9) and 96 threads (0.3-0.4) don't overlap, indicating a clear measured drop across the socket boundary. However, the sampling does not establish an abrupt change exactly when the second socket starts being used.

The measured peak was **2.6Mops/s at one thread**, matching my predicted peak of 1-2 threads. Throughput generally declined as thread count increased. At **128, 192, and 256 threads**, throughput remained **~0.3Mops/s**, so oversubscription produced no additional visible drop at the reported precision.

Part 3:

Explanation

The coarse-map throughput peaks at **2.6Mops/s with one thread** and generally declines as threads are added. Every operation acquires the same mutex, so the main contended cache line is the 64-byte line containing the mutex's lock word. With one thread, that line can remain in the executing core's cache. With multiple threads, accesses to the lock cause its ownership to move between cores.

With MESI, immediately after a core writes the lock word, its copy is **modified**, and other cores' copies are **invalid**. A read by another core can cause the line to become **shared**. When another core attempts a write or atomic update, it must obtain exclusive ownership and invalidate other copies (after the write, its copy = M). This transfers ownership of the whole cache line, even though the lock word occupies only part of it. Cache-line ownership is distinct from mutex ownership, where a failed atomic acquisition can generate coherence traffic without successfully acquiring the mutex.

From lecture, a two-core true-sharing experiment on Frontera measured **~12ns/op** or **40 cycles at 3.29GHz**. We use this as an estimate of a same-socket cache-line handoff, rather than a direct measurement of our mutex's handoff latency. This reflects the cost of transferring

ownership between cores instead of accessing a line already held locally. This lecture also measured a local L2 access at **~4.26 ns (14 cycles)**, compared with **~12 ns (40 cycles)** for the true-sharing experiment. Thus, equal L1-miss counts can represent substantially different costs depending on where the line comes from.

Adding threads makes throughput worse rather than flat because the operations remain serialized while their overhead increases. Competing threads generate ownership transfers, failed acquisition attempts, and mutex blocking and wakeup activity. Protected map data may also need to move between caches. Thus, extra threads add work without allowing simultaneous map operations. Throughput falls from **0.7Mops/s at 64 threads** to **0.4Mops/s at 96 threads**. This is consistent with additional overhead when both sockets participate, although the curve does not isolate its cause. At **128-256 threads**, throughput remains at **~0.3Mops/s**, so oversubscription produced no additional drop at the reported precision (consistent with previous observation).

CoarseMap performance counters - Frontera

Default workload: 80% find, 10% insert, 10% erase

Threads	Mops/s	Cycles / op	Instructions / op	IPC	L1 misses / op	LLC misses / op	HITM / op	Context switches / op
1	2.3	1,399	334.6	0.239	24.00	0.267	0.0001631	5.676e-08
8	0.8	17,965	4,727.3	0.263	131.35	27.659	11.24	0.5021
28	0.8	95,697	11,402.9	0.119	159.53	36.248	16.05	0.5435

Operations: T=1: 70,477,824; T=8: 23,984,128; T=28: 23,953,408

Per-op values = raw event counts / operations; IPC = instructions / cycles.

LLC: cache-misses. HITM: mem_load_i3_hlt_retired.xsnp_hitm (qualifying loads).

At **T=1**, there are **24 L1 misses/op**. A balanced tree containing **512K** nodes has a depth of **~19 levels**, so this miss count is plausible for pointer-based traversal combined with value access, insertion, erasure, and allocation. The entire tree is much larger than **Frontera's 1MB L2 cache**, but frequently accessed upper levels can remain cached. Furthermore, some L1 misses are served by L2/L3 rather than main memory (which is shown through low single-thread **0.267 LLC misses/op**, meaning L1 misses are being serviced elsewhere in the cache hierarchy).

Between **T=1** and **T=8**, cycles/op increased by a factor of **12.84x (from 1.4k to 18k)**, while **L1 misses** increased **5.47x (from 24 to ~130)**. Miss count alone therefore doesn't explain the increased CPU cost. An L1 miss served by local L2 has a different cost from a miss requiring data from another core that holds the line Modified. However, the slowdown cannot be attributed entirely to more expensive misses: **instr/op** also increased **14.13x (from 335 to 4.7k)**, showing substantial extra work under contention. **IPC** slightly rises from **0.239 to 0.263** at **T=8**.

HITM increases from **0.0002 (T=1) to 11 (T=8) to 16 (T=28)**. This is evidence of increased cache-to-cache transfers involving modified data. A simple model of one mutex handoff/op doesn't fully account for these counts. HITM measures qualifying loads throughout the measured execution, including access to protected data, rather than identifying only the mutex line or counting successful lock acquisitions. The results support coherence traffic as a contributor, but do not directly measure the latency of a handoff.

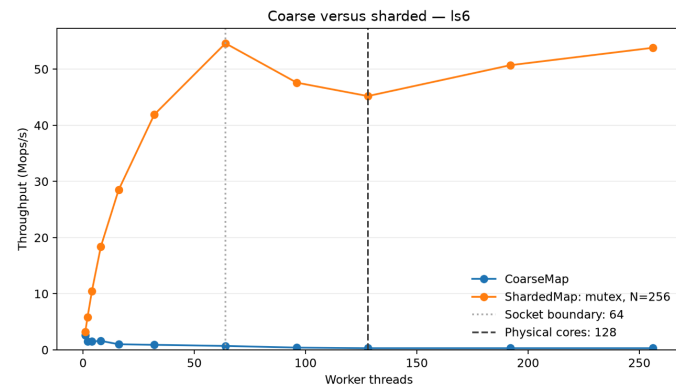
It went from **basically 0 (T=1) to >0.5 context switches/op (T=8, T=28; essentially one switch every 2 ops)**. These rates are consistent with substantial mutex blocking and wakeup activity, although the counter doesn't identify the cause of each switch. At **T=28**, **IPC falls to 0.119**, while going up to **96k cycles/op**. Additional threads still cannot execute map operations concurrently, but they add acquisition, coherence, and scheduling overhead. This explains why a global lock can make throughput worse rather than flat.

Part 4:

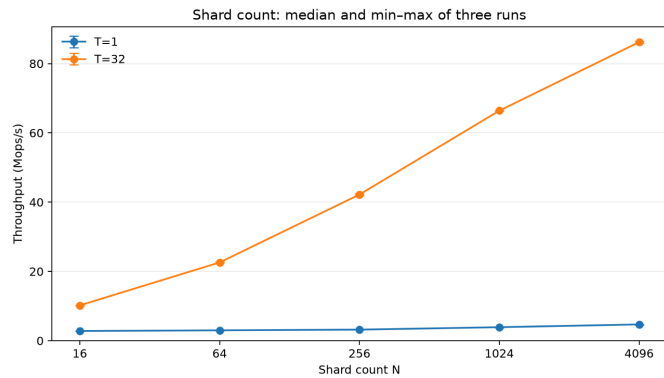
size() Explanation

size() acquires all shard locks in increasing index order and holds them while summing, giving an exact snapshot. Two concurrent **size()** calls cannot deadlock because they acquire locks in the same order. A concurrent insert either finishes before **size()** acquires its shard's lock or waits until **size()** releases it. Since insert holds only one lock, it cannot form a deadlock cycle. An approximate version that releases each shard before locking the next can combine counts from different times and return a total that never existed at any single instant.

Coarse vs Sharded Plot



Sharded CountPlot



At T=32, we choose 4,096 shards as it achieved 86.2Mops/s (the best out of the given range from 16-4096). Single-thread throughput also increased from 3.9 to 4.7Mops/s, showing that smaller trees contribute to the gain alongside reduced contention. This is the best shard count tested.

Numerical Collision Argument

Assuming independent, uniform shard choices, a pair collides at with probability $1/N$. At T=32 and N=4096, the expected number of colliding pairs is $\frac{T(T-1)}{2N} = 0.121$, and an operation shares its shard with another choice with probability $1 - (1 - \frac{1}{4096})^{31} = 0.754\%$. Single-thread throughput implies an uncontended cost of $\sim 213\text{ns/op}$. At T=32, average worker time = $\sim 371\text{ns/op}$. If we attribute the additional 158ns to collisions, the fitted collision penalty is $\frac{158\text{ns}}{0.00754} = 21\mu\text{s}$. Collision probability multiplied by this penalty is therefore 158ns, $\sim 74\%$ of uncontended cost. This is a fitted estimate that includes cache and scheduling effects, not a directly measured mutex-wait latency. Together with the throughput results, it supports choosing 4,096 shards as the best among the tested counts.

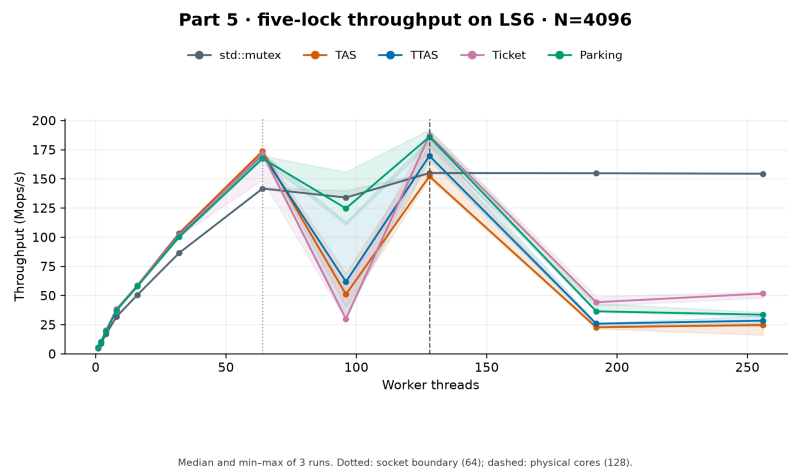
Part 5:

Correctness and Relaxed Ordering

TASLock, TTASLock, TicketLock, and ParkingLock passed the ordinary and TSan tests. Changing TAS to use relaxed ordering everywhere left the ordinary tests passing but caused TSan races in both the map and lock tests. The lock variable itself remains atomic, but relaxed ordering does not establish the acquire/release synchronization needed for the non-atomic data protected by the lock. A release unlock makes writes from the critical section visible before the lock is released, while a later acquire synchronizes with that release before the next thread accesses the protected data. Relaxed ordering provides no such ordering, which is why the ordinary tests can appear to work while TSan still detects incorrect synchronization.

Five-lock Throughput Sweep

Five-lock throughput on LS6, N=4096, 80/10/10 workload; median and min-max of three runs.



Median and min-max of 3 runs. Dotted: socket boundary (64); dashed: physical cores (128).

At T=128, mutex/TAS/TTAS/Ticket/Parking reach 155.1/152.3/169.8/187.4/186.0 Mops/s, respectively. At T=256, they reach 154.4/24.8/28.5/51.7/33.6 Mops/s. Custom locks are competitive before oversubscription but drop sharply afterward, while std::mutex remains stable. Parking performs better than TAS and TTAS after oversubscription, but still falls far below std::mutex.

One-shard Contention

Prediction: For the one-shard contention experiment, I expect TAS to have the most L1 misses per operation because every waiting thread repeatedly performs an atomic exchange on the same lock word, causing heavy cache-line ownership traffic. I expect TicketLock to have the highest instruction count because it performs additional ticket bookkeeping, repeatedly checks the serving counter, and periodically executes the yield path. The lock with the most instructions does not have to have the most misses because instruction count and coherence stalls measure different costs.

One-shard Contention on Frontera, T=8, N=1

Lock	Operations	Mops/s	Cycles/op	Instr/op	L1 misses/op	HITM/op	CS/op
Mutex	21,044,224	1.052	12,771	5,553	147.99	22.161	0.648
TAS	23,263,232	1.163	22,549	423	60.74	3.529	5.07e-06
TTAS	39,678,976	1.984	13,215	1,165	43.81	1.543	3.76e-06
Ticket	29,851,648	1.492	17,571	4,834	47.34	1.626	3.95e-06
Parking	25,947,136	1.297	18,367	4,356	98.07	13.402	0.0904

Each row uses the second repetition so that every normalized metric uses the same operation count. All three repetitions remain in the raw results.

The prediction was only partly supported. TAS did not have the most L1 misses overall: mutex had 147.99 misses/op and Parking had 98.07, compared with 60.74 for TAS. However, we observed the predicted TAS-versus-TTAS behavior. Looking at the table, TTAS reduced L1 misses/op from 60.74 to 43.81 and HITM/op from 3.529 to 1.543 while increasing throughput from 1.163 to 1.984 Mops/s. Mutex also had the highest instruction count at 5,553 instructions/op rather than the predicted TicketLock.

Why are TAS and TTAS different?

TAS repeatedly performs exchange() while waiting, so a waiter repeatedly requests writable ownership of the lock line, forcing other cached copies to be invalidated. With TTAS, waiting threads poll with loads, so their copies can remain in a readable/shared state while the lock stays held. On release, the owner's write invalidates those readable copies, and the waiters then compete for writable ownership. The backoff in TTAS spreads out these retries.

What is the spin hint for?

The processor spin hint makes the busy-wait loop easier on the processor, but it is not a memory fence and does not give the CPU to another thread

Why does TicketLock yield while TTAS does not?

TicketLock has a fixed FIFO successor. If the thread holding the next ticket is descheduled, later waiters cannot skip it even if they are currently running. Yielding after 64 spins gives other runnable threads a chance to execute. TTAS has no fixed successor, so any running contender can acquire the lock when it becomes free

Oversubscription on LS6, T=32 restricted to CPUs 0-7, N=4096

Lock	Operations	Mops/s	Cycles/op	Instr/op	L1 misses/op	CS/op	Busy estimate*
Mutex	549,586,944	27.441	934	333	18.59	8.04e-05	91.5%
TAS	118,515,712	5.912	4,696	356	26.50	0.000123	99.2%
TTAS	109,573,120	5.467	5,028	599	14.98	0.000134	98.2%
Ticket	247,819,264	12.386	1,829	588	18.04	0.0807	80.9%
Parking	141,248,512	7.057	3,862	1,976	25.21	0.000602	97.3%

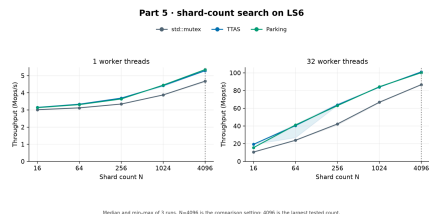
*Busy estimate = (cycles/op × Mops/s)/(8×3500), assuming an active frequency of 3.5 GHz. This is only an estimate because the task clock and actual active frequency were not saved.

TAS and TTAS complete only 5.912 and 5.467 Mops/s while their estimated busy fractions are near 99%, showing that the cores are doing substantial work while making little useful progress. Std::mutex reaches 27.441 Mops/s with far fewer cycles/op. TicketLock reaches 12.386 Mops/s and has much more context switching because it explicitly calls yield. Parking reaches 7.057 Mops/s and still has a high estimated busy fraction, so having a sleep path does not automatically make it behave like std::mutex

ParkingLock vs std::mutex

Parking is faster at T=128, reaching 186.0 Mops/s versus 155.1 for mutex, but at T=256 it falls to 33.6 Mops/s while mutex remains at 154.4. Our ParkingLock spins briefly before using atomic::wait/notify, but this is not enough to match the standard mutex once the system is oversubscribed. The measurements show the difference without requiring assumptions about the internal implementation of std::mutex.

Shard-count redo for TTAS and Parking



At T=32, TTAS increases from 63.84 Mops/s at N=256 to 101.14 at N=4096, while Parking increases from 62.97 to 100.22. We therefore keep N=4096.

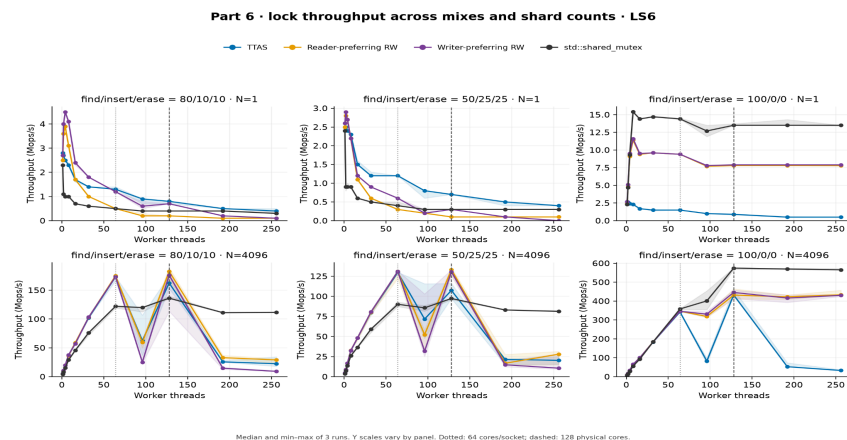
For T=32 and N=4096, $p=1-(1-1/4096)^{31} \approx 0.00754$. Under the independent uniform-choice model, an operation has only about a 0.75% chance of sharing its shard with another thread. TTAS's uncontended cost is 189.1 ns/op, versus 316.4 ns/op per worker at T=32. Attributing the additional 127.3 ns to collisions gives a fitted penalty of $127.3/0.00754 \approx 16.9 \mu s$. Probability $\times$ penalty is 127.3 ns, about 67.3% of the uncontended cost. Parking similarly gives 186.7 ns uncontended, 319.3 ns per worker, a fitted penalty of 17.6 μs , and an expected additional cost of 132.6 ns (71.0% of its uncontended cost). These fitted penalties include other concurrency overheads, rather than directly measuring lock-wait latency. We retain N=4096 as the best tested choice.

Part 6:

When do shared-reader locks beat TTAS?

RWLock and RWLockWP passed the ordinary tests and TSan.

Four locks $\times$ three mixes $\times$ N=1 and N=4096 on LS6; medians and min-max ranges.



At N=1, T=32, and 100/0/0, TTAS reaches 1.5 Mops/s, both custom RW locks reach 9.6, and std::shared_mutex reaches 14.7. With only one shard, TTAS serializes every read, while an RW lock allows multiple readers to access the shard simultaneously.

When do they not help?

At N=1, T=32, and 50/25/25, TTAS/RW/RW-WP/std::shared_mutex reach 1.2/0.6/0.9/0.5 Mops/s. Reader entry and exit still require atomic updates to the reader count, while writers require exclusive access. With more writes, the benefit of overlapping readers shrinks while the bookkeeping remains.

At N=4096, T=32, and 80/10/10, TTAS/RW/RW-WP reach 102.0/103.1/103.4 Mops/s. These results are very close because heavy sharding already allows unrelated operations to proceed concurrently, leaving much less for shared-reader locking to gain.

Where does std::shared_mutex fall?

At N=4096 and T=128 with 100% reads, std::shared_mutex reaches 574.5 Mops/s compared with 432.6 for TTAS, showing a large benefit from concurrent readers. Its much lower one-shard throughput once writers are introduced shows the cost of reader/writer coordination: when a writer is waiting for exclusive access, reader admission and wakeup behavior can substantially reduce throughput. However, aggregate throughput alone is not enough to determine the library's exact fairness or writer-preference policy.

What effect does writer preference have?

Reader and writer throughput on LS6, 28 readers and 4 writers

N	Lock	Readers Mops/s	Writers Mops/s	Total Mops/s
---	------	----------------	----------------	--------------

1	Reader-preferring	8.1	0.0	8.118
1	Writer-preferring	0.0	0.5	0.531
4096	Reader-preferring	109.8	10.6	120.389
4096	Writer-preferring	109.8	10.6	120.346

At N=1, writer preference shifts progress away from readers and toward writers. Total throughput falls from 8.118 to 0.531 Mops/s. The printed 0.0 values mean less than 0.05 Mops/s at the output precision, and stem from rounding

At N=4096, both versions are almost identical: roughly 109.8 Mops/s of reader throughput and 10.6 Mops/s of writer throughput. Sharding reduces the contention that makes admission preference important.

Part 7:

Correctness, bucket mapping, and shared memory

StripedHashMap passed the ordinary and TSan tests, including the 7-bucket/3-stripe case.

The implementation first hashes the key into one of B buckets and assigns contiguous bucket ranges to L stripes with the equation: $\text{bucket} = \text{hash}(\text{key}) \% B$, with the stripe determined from the bucket's contiguous range. For B=7 and L=3, the ranges contain 3, 3, and 1 buckets.

Padding separates the stripe locks, but bucket-head pointers can still share cache lines across stripe boundaries, and heap-allocated nodes can also share lines. Padding therefore eliminates lock-to-lock false sharing, but not every possible source of cache-line sharing.

Hash table versus sharded tree

Part 7 · hash-table / tree crossover · LS6 · T=1, N=L=4096

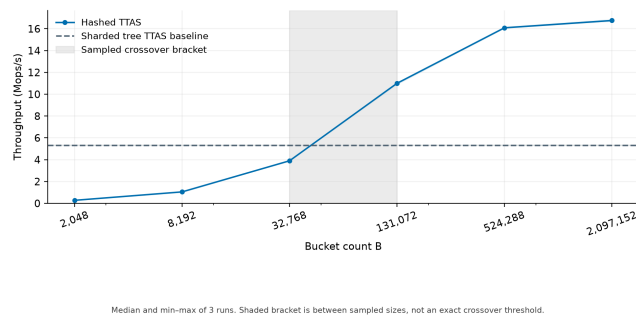


Figure: T=1, N=L=4096; hash bucket-count sweep against the sharded-tree baseline.

The tree reaches 5.324 Mops/s. The hash table reaches 16.756 Mops/s at B=2,097,152, 11.000 at B=131,072, 3.898 at B=32,768, and 0.274 at B=2,048. The measured crossover therefore falls between 32,768 and 131,072 buckets. With many buckets, hash chains are short, and lookups avoid the comparisons and dependent accesses of a tree traversal. As the bucket count decreases, chains become longer, and serial linked-list traversal removes that advantage

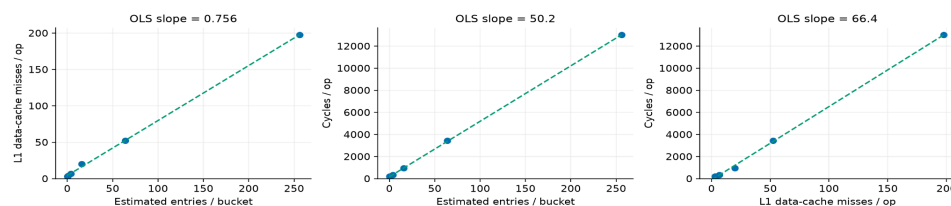
L1 misses versus chain length

Prediction: As mean chain length increases, I expect L1 misses per operation to increase roughly linearly because each additional node visited in the linked list can require another cache-line access. There should also be a fixed cost from accessing the bucket head, so I expect something of the form: $\text{L1 misses/op} \approx C + k(\text{mean chain length})$, with a slope below or around one because successful searches may stop before reaching the end of the chain.

Also, we assume that $\text{mean chain length} = 524288/B$, using an estimated resident population of 524,288 entries.

L1 misses/op versus estimated mean chain length

Part 7 · chain length and counter cost · LS6 · T=1, L=4096



Six single-run counter points; occupancy approximated as 524,288. Descriptive fits; cycles/miss slope is not isolated miss latency.

Bucket-count counters on LS6, T=1, L=4096

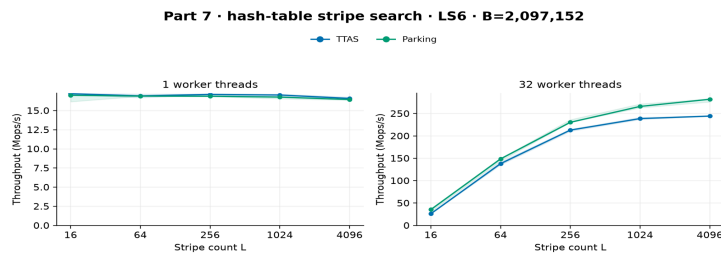
Buckets	Estimated entries/bucket	Operations	L1 misses/op	Cycles/op
2,048	256	5,361,664	197.657	13,018.8
8,192	64	20,272,128	52.391	3,445.9
32,768	16	73,152,512	20.034	954.8
131,072	4	212,144,128	6.817	328.5
524,288	1	321,734,656	3.845	217.4
2,097,152	0.25	340,339,712	3.052	205.4

The measured slope is 0.756 additional L1 misses/op per additional estimated entry per bucket. This agrees with the prediction that misses increase roughly with chain length. A slope below one is reasonable because successful searches may stop partway through a chain, while unsuccessful searches may traverse the whole chain. Bucket-head and lock accesses add a roughly fixed cost, and caching also affects the measured slope.

For the long chains, cycles/op versus misses/op gives about 66 cycles per additional L1 miss. Using only the two longest-chain measurements gives about 65.9 cycles/miss, or approximately 18.8 ns at 3.5 GHz. This is much larger than an L1 or typical L2 access and is most consistent with an LLC/L3 or other beyond-L2 access. However, the estimate also includes pointer traversal and other execution costs, so it should not be interpreted as the exact latency of a single LLC access.

Stripe-count selection

TTAS and Parking, T=1 and T=32, B=2,097,152.



Median and min-max of 3 runs. These results tune L separately from the fixed N=4096 comparison setting.

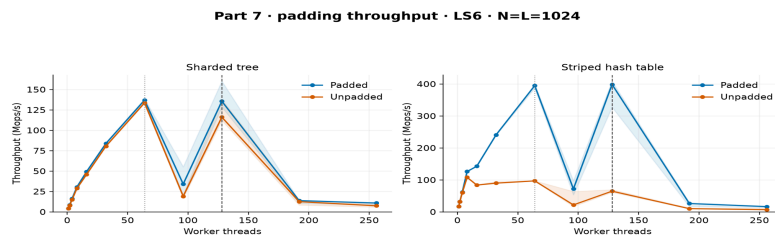
At T=32, TTAS increases from 238.81 Mops/s at L=1024 to 244.39 at 4096; Parking increases from 265.98 to 282.00. We select L=4096, the best among the tested counts, with diminishing gains and a small T=1 cost.

At T=32 and L=4096, $p=1-(1-1/4096)^{31} \approx 0.00754$: about 0.754% expected same-stripe overlap under independent uniform choices. TTAS's uncontended cost is 60.28 ns/op, versus 130.94 ns/op per worker at T=32. Attributing the additional 70.66 ns to collisions gives a fitted penalty of $70.66/0.00754 \approx 9.37 \mu s$. Probability $\times$ penalty is 70.66 ns, about 117.2% of the uncontended cost. Parking similarly gives 60.81 ns uncontended, 113.47 ns per worker, a fitted penalty of 6.98 μs , and an expected additional cost of 52.66 ns—86.6% of its uncontended cost. These fitted penalties include other concurrency overheads, rather than directly measuring lock-wait latency. We retain L=4096 as the best tested choice.

Padding

Prediction: TTASLock is one byte in this build, so up to 64 unpadded TTAS locks can fit on one 64-byte cache line. I expect padding to make little difference at low thread counts but become more important at higher thread counts, where multiple cores are more likely to modify different locks that happen to share the same cache line.

Padded versus unpadded sharded tree and striped hash table, N=L=1024



Median and min-max of 3 runs. Dotted: 64 cores/socket; dashed: 128 physical cores.

The largest hash-table difference occurs at T=128: padded throughput is 398.7 Mops/s versus 64.9 Mops/s unpadded, a 333.8 Mops/s gain and 6.14× speedup. The ranges are 327.6–408.5 padded and 61.5–69.4 unpadded, so they do not overlap.

For the sharded tree, the largest difference is also at T=128: 135.7 versus 116.2 Mops/s. Its ranges overlap, so this smaller difference is less clearly separated from run-to-run variation.

The much larger hash-table effect makes sense because many one-byte unpadded stripe locks can occupy the same cache line. Threads can therefore invalidate each other's cache-line copies even when they are locking completely different stripes. A tree shard contains substantially more metadata, so its locks are not packed nearly as densely. The benefit from padding is bounded because lock false sharing is only one part of the operation cost; tree/hash traversal, pointer chasing, genuine same-stripe contention, bucket-head sharing, memory bandwidth, and scheduling overhead remain even after the lock lines are separated.

Did HITM confirm the padding prediction?

Padding counters on Frontera, T=28, L=1024

Layout	Operations	Mops/s	Cycles/op	L1 misses/op	HITM/op
Padded	5,189,678,080	259.328	353.7	5.236	1.168
Unpadded	1,438,157,824	71.904	1276.8	6.118	0.835

Across all three runs, median L1 misses/op decrease from 6.094 unpadded to 5.228 padded, while HITM/op increases from 0.835 to 1.168. Throughput ranges are 71.56–72.04 unpadded and 258.08–259.43 padded.

The throughput results clearly show a large padding benefit, and L1 misses also move in the expected direction. However, HITM/op does not decrease with padding. The counters therefore did not uncover a difference that throughput failed to resolve, because the throughput gap was already very large. This particular HITM event counts qualifying loads across the whole workload rather than only lock-line ownership transfers, so it does not by itself explain the padding speedup or determine what fraction of each operation's time was spent on lock-line coherence.

Compute + TSan

Part 3 used CPU-number order, spanning both sockets at T=8 and T=28. Part 5 contention used even CPUs 0–14, and Part 7 padding used even CPUs 0–54, keeping both experiments on socket 0.

```
c310-005.ls6.tacc.utexas.edu
g++ (GCC) 11.2.0
#define HAVE_SHARDED 1 // Part 4: ShardedMap (concurrent_map.h)
#define HAVE_LOCKS 1 // Part 5: TAS/TAS/Ticket/Park (locks.h)
#define HAVE_RW 1 // Part 6: RWLock, RWLockWP (locks.h)
#define HAVE_HASHED 1 // Part 7: StripedHashMap (hash_map.h)
TSAN_OPTIONS=halt_on_error=1 ./test_map_tsan
all checks passed
TSAN_OPTIONS=halt_on_error=1 ./test_locks_tsan
all checks passed
make=0 test=0
```

```
ls6-005-11013018@ls6:
ls6-005-11013018$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/passwd
daemon:x:2:2:daemon:/usr/sbin:/usr/sbin/passwd
adm:x:3:3:adm:/var/adm:/usr/sbin/passwd
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/passwd
sync:x:5:0:sync:/bin:/bin/sync
shutdown:x:6:6:shutdown:/bin:/usr/sbin/passwd
halt:x:7:7:halt:/bin:/usr/sbin/passwd
mail:x:8:8:mail:/var/mail:/usr/sbin/passwd
news:x:9:9:news:/var/news:/usr/sbin/passwd
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/passwd
operator:x:11:11:operator:/root:/usr/sbin/passwd
man:x:12:12:man:/var/man:/usr/sbin/passwd
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/passwd
mysql:x:1234:1234:mysql:/var/lib/mysql:/bin/false
postgres:x:12345:12345:postgres:/var/lib/postgresql:/bin/false
```

```
ls6-005-11013018@ls6:
ls6-005-11013018$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/passwd
daemon:x:2:2:daemon:/usr/sbin:/usr/sbin/passwd
adm:x:3:3:adm:/var/adm:/usr/sbin/passwd
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/passwd
sync:x:5:0:sync:/bin:/bin/sync
shutdown:x:6:6:shutdown:/bin:/usr/sbin/passwd
halt:x:7:7:halt:/bin:/usr/sbin/passwd
mail:x:8:8:mail:/var/mail:/usr/sbin/passwd
news:x:9:9:news:/var/news:/usr/sbin/passwd
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/passwd
operator:x:11:11:operator:/root:/usr/sbin/passwd
man:x:12:12:man:/var/man:/usr/sbin/passwd
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/passwd
mysql:x:1234:1234:mysql:/var/lib/mysql:/bin/false
postgres:x:12345:12345:postgres:/var/lib/postgresql:/bin/false
```

Data from [sweep.sh](https://www.sweep.sh) and lscpu:

Frontera SoC: cores=56; sockets=2; cores/socket=28; pin order = CPU-numbered: even on socket 0, and odd on socket 1

LS6 SoC: cores=128; sockets=2; cores/socket=64; pin order: 0-63 on socket 0 and 64-127 on socket 1

Contributions: Tu primarily worked on Parts 1–4, including the coarse and sharded map implementations and their associated measurements and analysis. Rayaan primarily worked on Parts 5–7, including the custom locks, reader-writer locks, striped hash table, and their associated measurements and analysis; he also confirmed N=4096 as the best-tested shard count (from the README) for the measurements used in Parts 5–7. We worked on the overall implementation in parallel, discussed design and debugging decisions together, and reviewed each other's work. Additionally, we used AI assistance to create scripts to plot data collected from LS6 and Frontera.